基于差分序列的轻量级 Transformer 预测模型

1 核心思路

本方案的核心在于将金融时间序列预测问题转化为对**价格增量**（一阶差分）的预测。给定连续 14 天的价格 $p_{t-13}, ..., p_t$，目标不是直接预测 p_{t+1}，而是预测其与 p_t 的差值 $\Delta_{t+1} = p_{t+1} - p_t$。此转换简化了任务，并使输入数据更具平稳性。

同时, 本方案利用多头注意力与位置编码结合的机制, 成功让模型关注序列元素的关联, 拥有很好的预测效果.

2 数据预处理

1. **固定范围归一化**：为避免信息泄露，所有价格数据使用固定的区间 $[0, 300]$ 进行 Min-Max 归一化：
$$\hat{p}_i = \frac{p_i}{300}$$
2. **生成差分输入**：对于每个 14 天的窗口，计算其一阶差分，得到一个 13 维的向量作为模型输入：
$$\mathbf{x}_{diff} = [\hat{p}_{t-12} - \hat{p}_{t-13},\ \hat{p}_{t-11} - \hat{p}_{t-12},\ \ldots,\ \hat{p}_t - \hat{p}_{t-1}]$$

3 模型结构 (`MiniTransformerNet`)

模型结构极为精简，包含以下关键组件：

1. **嵌入层**：输入为 13 维差分向量。嵌入层通过一个线性变换，将每个时间步的标量差分值独立地映射到一个 32 维的嵌入向量。具体而言，13 维输入经过线性层后变为 $13 \times 32 = 416$ 维，再被重塑为形状 $(13, 32)$ 的序列，其中 13 是序列长度，32 是每个位置的特征维度（即嵌入维度）。

2. **位置编码**：由于 Transformer 架构本身不具备序列顺序的概念，必须显式地注入位置信息。本模型采用 Vaswani 等人提出的正弦/余弦绝对位置编码：
$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \tag{1}$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \tag{2}$$

其中，pos 是时间步的位置索引（$0, 1, ..., 12$），i 是嵌入维度中的索引（$0, 1, ..., d_{\text{model}}/2-1$），$d_{\text{model}} = 32$ 是嵌入维度。这种编码方式能够为模型提供关于序列中元素相对距离的信息，并且其周期性特性有助于模型泛化到比训练时更长的序列。

3. **自注意力层**：这是模型的核心，旨在动态地学习序列中不同时间步之间的依赖关系。给定嵌入序列 $\mathbf{X} \in \mathbb{R}^{13\times 32}$，首先通过三个可学习的权重矩阵 $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V \in \mathbb{R}^{32\times 32}$ 投影得到查询（Query）、键（Key）和值（Value）矩阵：

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}^K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}^V \tag{3}$$

缩放点积注意力的计算公式如下：

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V} \tag{4}$$

其中，$d_k = 32/4 = 8$ 是每个注意力头的维度。softmax 函数确保了注意力权重之和为 1，而缩放因子 $\sqrt{d_k}$ 则防止了点积过大导致梯度消失的问题。该机制允许模型在预测未来增量时，自适应地关注历史序列中最具相关性的部分。

为了增强模型的表达能力，本方案采用**多头注意力**（Multi-head Attention）。它将上述过程并行执行多次（本模型中为 4 次），并将结果拼接起来：

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, ..., \text{head}_h)\mathbf{W}^O \tag{5}$$

其中每个 $\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V)$。多头机制使模型能够在不同的表示子空间中共同关注来自不同位置的信息。

4. **池化与输出**：对自注意力层的输出在时间维度上进行平均池化，然后通过一个小型 MLP（32→64→1）直接输出预测的增量 $\hat{\Delta}_{t+1}$。

该设计舍弃了复杂的前馈网络和残差连接，仅保留 Transformer 的核心——自注意力机制，实现了模型的轻量化。

4 预测流程

预测函数 `predict` 的执行步骤如下：

1. 对输入的 14 天原始价格进行固定范围归一化。
2. 计算归一化后价格的一阶差分，得到 13 维输入向量。
3. 将输入向量送入训练好的 `MiniTransformerNet` 模型，得到预测的增量 $\hat{\Delta}_{t+1}$。
4. 通过公式 $\hat{p}_{t+1} = \hat{p}_t + \hat{\Delta}_{t+1}$ 重建归一化尺度下的预测价格。
5. 对预测价格进行逆归一化，得到最终的原始价格预测值。

5 实验结果

模型在系统给定的测试下评估，性能指标如下：

图 1: 预测性能指标

模型在验证集上的预测和实际结果对比如下:

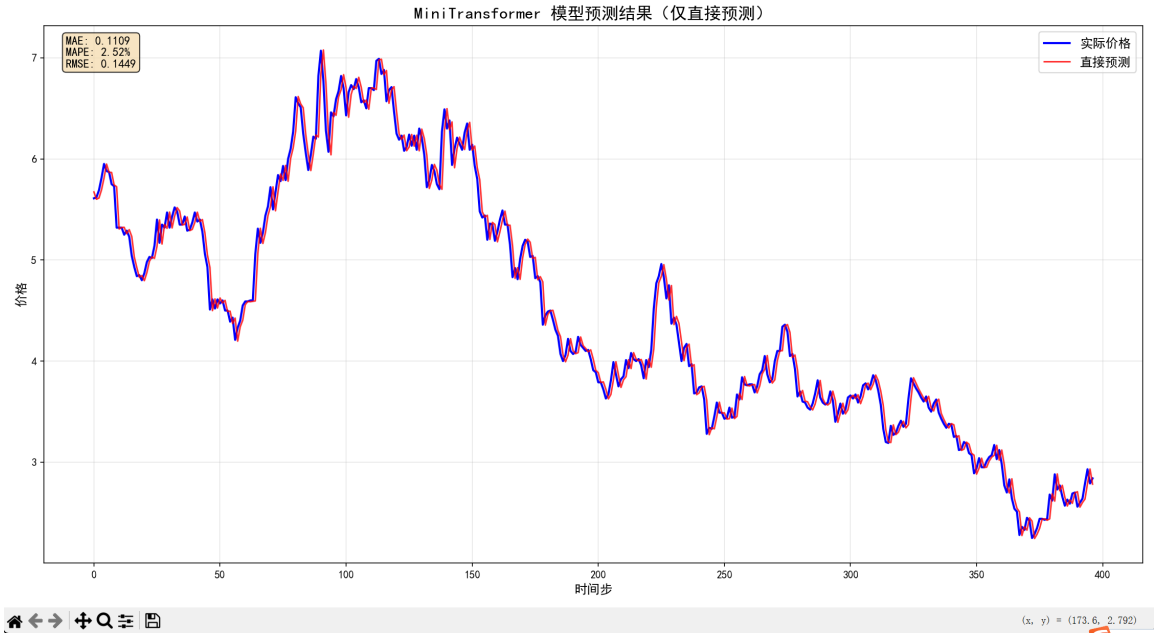

图 2: 预测与真实结果对比

结果表明，该轻量级模型能够以极高的精度完成 14 天预测第 15 天价格的任务。

6 创新设计

本方案主要创新点在于在基础的注意力机制基础上, 采用差分预测和位置编码. 如果用无位置编码的原始数据, 结果如下:

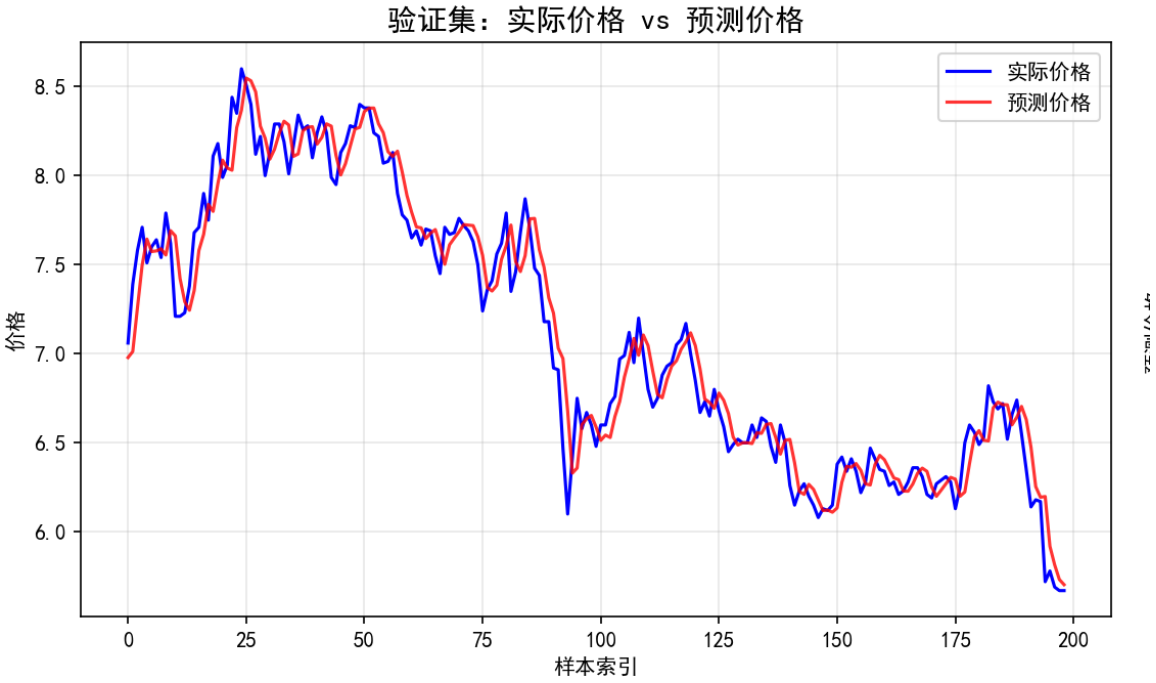

图 3: 无差分和位置编码预测与真实结果对比

可以看到, 此时模型在预测时出现明显的平滑, 不敢拟合大波动的点, 同时整体存在比较严重的滞后现象.

产生这样现象的原因如下:

首先，**差分预测**从根本上改变了任务的性质。原始价格序列通常具有强烈的趋势性和非平稳性，直接建模难度极大。通过预测增量 Δ_{t+1}，我们将问题转化为对围绕零均值附近波动的平稳序列进行建模。这极大地降低了模型的学习难度，并使其能够更敏锐地捕捉到价格的瞬时变化和突变点，从而避免了传统模型常见的过度平滑问题。

其次，**位置编码**是 Transformer 应用于时间序列任务的关键。如前所述，自注意力机制本身是置换不变的（Permutation-invariant），这意味着打乱输入序列的顺序不会改变输出。然而，时间序列的核心恰恰在于其严格的时序性。通过引入位置编码 $PE_{(pos)}$，我们将绝对位置信息 pos 注入到每个时间步的嵌入向量中，使得模型能够明确区分“昨天”和“前天”的信息。这对于消除预测中的滞后效应至关重要，因为模型可以学会赋予最近的时间步更高的权重。

最后，**多头自注意力机制**提供了强大的关系建模能力。不同于 RNN 或 CNN 等局部感受野的模型，自注意力机制允许序列中的任意两个时间步直接交互。在金融场景中，

这意味着模型可以学习到远距离的依赖关系，例如，识别出当前的价格波动模式与数周前某个特定事件后的模式高度相似，从而做出更准确的预测。

综上所述，差分预测解决了“预测什么”的问题，使其更契合金融数据的特性；位置编码解决了“何时发生”的问题，保证了时序信息不丢失；而自注意力机制则解决了“如何关联”的问题，实现了全局上下文感知。三者相辅相成，共同构成了本方案高效、精准的预测能力。

根据测试, 修改后 mape 从 0.0255 提升到了 0.0208,mae 从 0.884 提升到了 0.5827, 获得了大约 1/3 的性能提升, 是非常显著的效果.

7 方案对比

首先是 MLP 方案: 采用 14-128-64-32-1 的层数堆叠,MLP 的方案 MAE 高达 1.9684,MAPE 高达 0.28, 下面是结果:

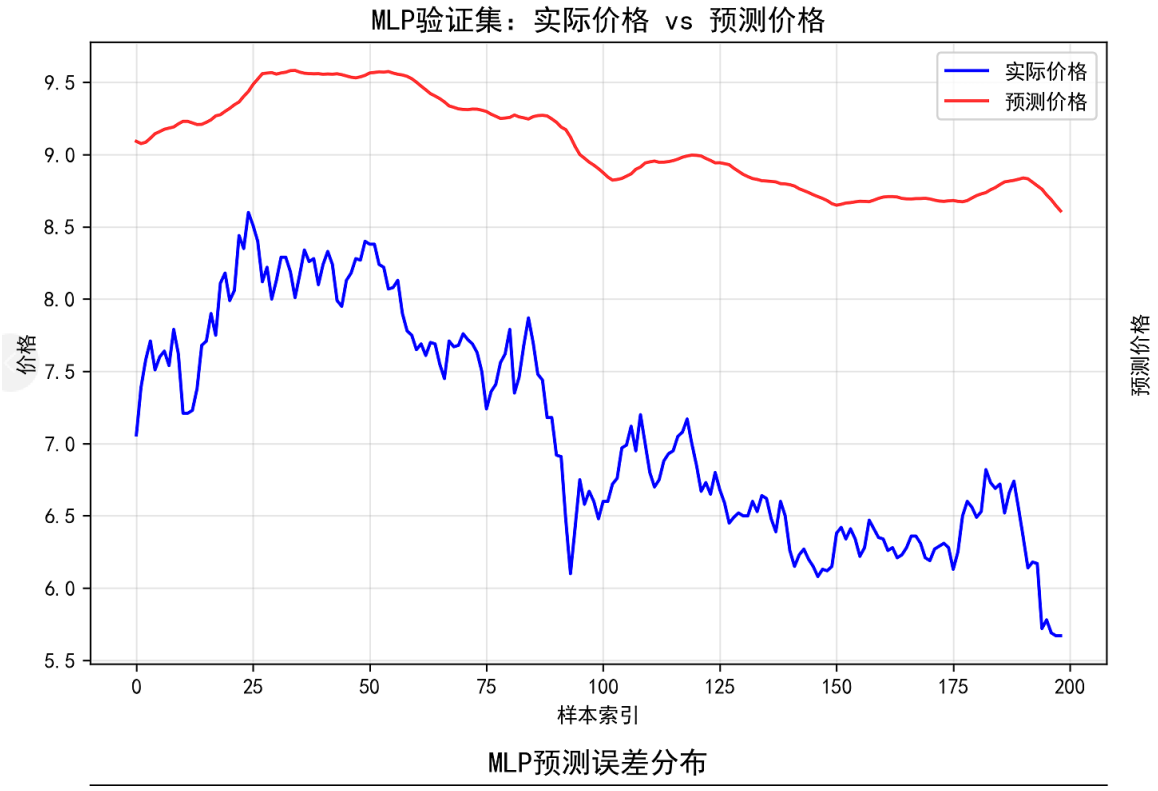

图 4: MLP 预测与真实结果对比

可以看到多层感知机不仅无法拟合模型的剧烈动荡, 还出现了严重的整体偏移, 不是一个好的方案.

下一个是一维卷积神经网络方案:

采用 1,3,5,7,9,14 多个尺度卷积核进行多尺度信息提取, 预测, 最终 MAE 为 1.75,MAPE 为 0.25, 略好于 MLP 方案, 但还是不够用, 下面是结果对比图:

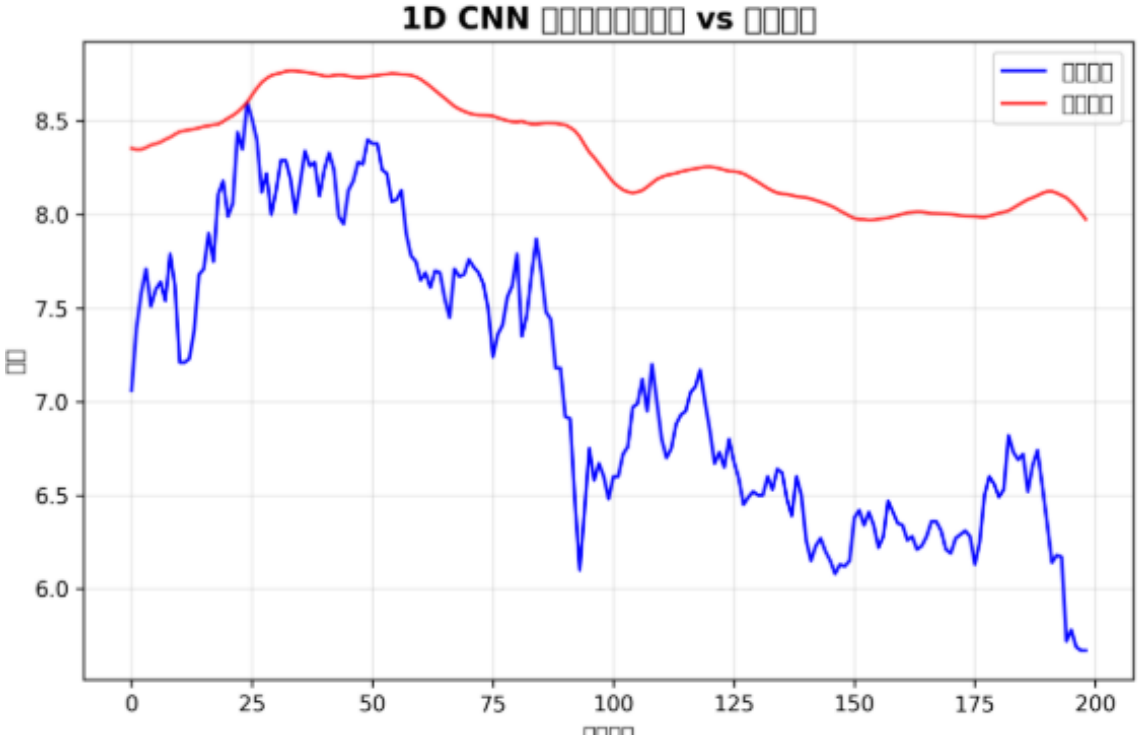

图 5: MLP 预测与真实结果对比

可以看到, 模型只能预测整体的大体趋势, 对于剧烈变化却无能为力, 虽然比 mlp 性能更好但也远远不足.

8 总结

本次 A 股预测实验中, 设计了完整的基于轻量的 transformer 结构的单步预测模型, 并对比了传统的 MLP,1D-CNN 等模型, 同时将原始数值预测改成差分预测, 最终取得了良好的预测效果.

具体代码实现见 train.py 文件